

PORTADA

Análisis y Diseño de Sistemas II

Semana 12 — Patrones GoF Creacionales II: Builder, Prototype, Singleton, y Laboratorio de implementación

Universidad de Antioquia · Ingeniería de Sistemas



APERTURA

**Un constructor con doce parámetros
no es un objeto complejo — es un objeto mal construido**

TRANSICIÓN

Cerrando la familia de creacionales GoF

El martes vieron **Factory Method** y **Abstract Factory**, los dos primeros patrones creacionales del catálogo GoF (Gang of Four) — soluciones concretas y documentadas para problemas recurrentes de creación de objetos, un nivel más específico que los patrones GRASP de la semana 10. Hoy completan la familia con los tres patrones creacionales que faltan: **Builder**, **Prototype** y **Singleton**.

Con estos cinco (Factory Method, Abstract Factory, Builder, Prototype, Singleton) se cierra el catálogo completo de creacionales GoF. La segunda mitad de la sesión es el **laboratorio de implementación**: cada equipo aplica al menos uno de los cinco sobre su propio proyecto de Fábrica Escuela, con código real.

CONTRAEJEMPLO

El problema: un constructor que crece sin control

```
class Pedido:
    def __init__(self, cliente, producto=None, cantidad=None,
                  descuento=None, direccionEnvio=None, notas=None,
                  metodoPago=None, cuponRegalo=None, prioridad=None):
        # nueve parámetros, la mayoría opcionales
        ...

# para usarlo sin varios de esos datos, hay que pasar None
# explícito en cada uno de los que no aplican:
pedido = Pedido("Ana", "Teclado", 1, None, None, None, "tarjeta", None, None)
```

A medida que `Pedido` gana parámetros opcionales, cada llamada al constructor se vuelve ilegible: hay que contar posiciones para saber qué `None` corresponde a qué dato. La alternativa de crear una combinación de **constructores sobrecargados** — uno por cada subconjunto de datos usado en la práctica — tampoco escala: el número de combinaciones crece más rápido que el número de parámetros.

CONCEPTO

Builder: separar la construcción de la representación

Builder (Erich Gamma et al., "Design Patterns", 1994) separa el proceso de construir un objeto complejo — con muchos parámetros opcionales o varios pasos de configuración — de su representación final, permitiendo construir el mismo proceso paso a paso y terminar con distintas configuraciones del objeto.

En vez de un constructor gigante o varias combinaciones sobrecargadas, un objeto **Builder** aparte expone un método por cada dato configurable, cada uno retornando el propio builder — a este estilo se le llama **interfaz fluida** o **encadenamiento de métodos** (method chaining) — y un método final que entrega el objeto ya construido.



CONCEPTO

La solución: un Builder encadenable

```
class PedidoBuilder:
    def conCliente(self, cliente):
        self.pedido.cliente = cliente
        return self          # retorna self: encadenable

    def conProducto(self, producto, cantidad):
        self.pedido.agregarLinea(producto, cantidad)
        return self

    def construir(self):
        return self.pedido

# uso:
pedido = (PedidoBuilder()
        .conCliente(cliente)
        .conProducto(productoA, 2)
        .construir())
```

Cada llamada solo aparece si el dato aplica — no hay que pasar `None` explícito para los pasos que no se usan, y el orden de las llamadas no importa mientras `construir()` sea la última.

CONCEPTO · ACLARACIÓN IMPORTANTE

¿Y valores por defecto en el constructor no resuelven lo mismo?

En lenguajes que soportan parámetros con valor por defecto (ej. `descuento=0`), es tentador pensar que eso ya resuelve el problema del constructor con muchos parámetros. Resuelve **parte**: evita el `None` explícito, pero no resuelve la **legibilidad posicional** ni los **pasos de construcción con lógica propia** (ej. `agregarLinea` valida stock antes de aceptar un producto).

Builder también aporta algo que los valores por defecto no dan: un **objeto separado** que puede acumular estado de construcción a través de varias líneas de código, e incluso ser reutilizado para construir varias variantes del mismo objeto base — algo que un constructor, con o sin valores por defecto, no permite.

CONCEPTO · CUÁNDO USARLO

Cuándo vale la pena un Builder — y en qué se diferencia de Factory Method

- Un objeto con **varios parámetros opcionales** que rara vez se usan todos juntos.
- Un proceso de construcción con **pasos o validaciones intermedias**, no solo asignación directa de atributos.
- Se necesita que el mismo proceso de construcción pueda producir **representaciones distintas** de un objeto (ej. exportar el mismo pedido en un formato u otro).

Diferencia con Factory Method (martes): Factory Method resuelve *qué clase concreta* instanciar cuando hay varias posibles; Builder resuelve *cómo ensamblar* un objeto complejo paso a paso, sin importar si al final hay una sola clase o varias. Son problemas distintos y pueden combinarse: una fábrica puede usar un builder internamente.

CONCEPTO

Prototype: crear clonando, no instanciando desde cero

Prototype crea nuevos objetos clonando una instancia existente (el **prototipo**) en lugar de instanciar la clase desde cero, dando a cada objeto un método — habitualmente `clonar()` o `clone()` — que retorna una copia de sí mismo.

Es útil cuando crear un objeto desde cero es **costoso** (ej. requiere consultar una base de datos o realizar cálculos pesados para dejarlo en su estado inicial) o cuando se necesita **duplicar** un objeto ya configurado, con pequeñas variaciones sobre esa copia, en lugar de repetir toda la configuración desde el principio.



CONCEPTO

El método clonar()

```
class ConfiguracionReporte:
    def __init__(self, formato, filtros, columnas):
        self.formato = formato
        self.filtros = list(filtros)
        self.columnas = list(columnas)

    def clonar(self):
        return ConfiguracionReporte(self.formato, self.filtros, self.columnas)

# uso:
base = ConfiguracionReporte("PDF", filtrosEstandar, columnasEstandar)
variante = base.clonar()
variante.formato = "Excel"
```

`variante` nace como copia exacta de `base` , y solo se le cambia el atributo que sí necesita variar — el resto de la configuración (filtros, columnas) no se vuelve a construir desde cero.

CONCEPTO · ACLARACIÓN IMPORTANTE

Clonar no siempre es tan simple como copiar

Si un atributo del objeto es a su vez una lista u otro objeto (como `filtros` y `columnas` en el ejemplo anterior), `clonar()` debe decidir cómo copiarlo:

- **Copia superficial (shallow clone):** se copia solo la referencia a esa lista — el original y la copia terminan **compartiendo** la misma lista en memoria.
- **Copia profunda (deep clone):** se crea una lista nueva con los mismos valores — modificar la copia **no afecta** al original.

El código de la diapositiva anterior usa `list(...)` precisamente para forzar una copia profunda de esas listas. Si se hubiera copiado solo la referencia, modificar los filtros de `variante` también habría modificado silenciosamente los de `base` — un error difícil de detectar.

CONCEPTO · CUÁNDO USARLO

Cuándo vale la pena Prototype

- Crear el objeto desde cero implica un **costo real**: consulta a base de datos, cálculo pesado, o inicialización con muchos pasos.
- Se necesitan **varias variantes** de un objeto ya configurado, cambiando solo algunos atributos sobre una base común.
- La clase concreta que hay que copiar **no siempre se conoce de antemano** en el código que hace la copia — se clona lo que se tenga a mano, sin preguntar de qué clase exacta es.

Diferencia con Factory Method y Abstract Factory (martes): esos dos patrones crean instancias **nuevas desde cero**, decidiendo qué clase concreta usar; Prototype no decide una clase — **copia** una instancia que ya existe y ya está configurada.

CONCEPTO

Singleton: una única instancia accesible globalmente

Singleton garantiza que una clase tenga una **única instancia** en todo el sistema y provee un **punto de acceso global y controlado** a esa instancia — ningún otro código puede crear una segunda instancia de esa clase.

Se usa típicamente cuando existe un recurso que por definición debe ser único y compartido: la configuración global de la aplicación, un registro central de logs, o el acceso a un recurso que no admite múltiples instancias simultáneas.



CONCEPTO

El Singleton clásico: getInstance() estático

```
class ConfiguracionApp:
    _instancia = None

    def __init__(self):
        if ConfiguracionApp._instancia is not None:
            raise Exception("Usar getInstance(), no el constructor")
        self.valores = cargarConfiguracion()

    @staticmethod
    def getInstance():
        if ConfiguracionApp._instancia is None:
            ConfiguracionApp._instancia = ConfiguracionApp()
        return ConfiguracionApp._instancia

# uso, desde cualquier parte del código:
config = ConfiguracionApp.getInstance()
```

La primera llamada a `getInstance()` crea la única instancia; cualquier llamada posterior, desde cualquier parte del sistema, recibe esa misma instancia ya creada.

TENSIÓN CONCEPTUAL

Singleton es de los patrones GoF más criticados hoy

A diferencia de Builder o Prototype, Singleton tiene una reputación mucho más discutida en la práctica moderna. El problema no es la idea de "instancia única" en sí, sino cómo `getInstancia()` la entrega: queda como **estado global** accesible desde cualquier punto del código, sin que quede explícito en la firma de ninguna clase que la está usando.

- **Dificulta las pruebas unitarias:** el estado de la instancia única persiste entre pruebas si no se reinicia explícitamente, y no hay forma sencilla de reemplazarla por una versión de prueba.
- **Esconde dependencias:** una clase que llama a `ConfiguracionApp.getInstancia()` en medio de su código no declara esa dependencia en su constructor — hay que leer el cuerpo del método para descubrirla.
- **Si se usa para "todo":** termina funcionando como una bolsa global de estado compartido, lo que viola la cohesión de responsabilidad vista en la semana 7.



CONCEPTO · ALTERNATIVA MODERNA

La alternativa: ciclo de vida único, gestionado por fuera

Por estas críticas, muchos equipos hoy prefieren lograr "una sola instancia compartida" con **inyección de dependencias** (DI, técnica vista junto con DIP en la semana 9) en lugar de un Singleton clásico: se crea una única instancia del objeto en un punto central de arranque de la aplicación, y esa misma instancia se **inyecta por constructor** a cada clase que la necesita.

La diferencia clave: con DI la dependencia queda **explícita** en el constructor de cada clase — se puede ver, y se puede reemplazar por una versión de prueba en un test — mientras que con `getInstancia()` estático la dependencia queda **oculta** dentro del cuerpo del método. El resultado (una sola instancia compartida) es el mismo; lo que cambia es qué tan visible y reemplazable es esa dependencia.

Esto no invalida a Singleton — sigue siendo razonable en casos simples, sin necesidad real de aislarlo en pruebas — pero conocer la controversia evita aplicarlo por defecto sin pensarlo.

INTERACCIÓN

¿Cuál de los cinco patrones creacionales aplica?

En el chat, para cada caso indiquen cuál de los cinco patrones creacionales (Factory Method, Abstract Factory, Builder, Prototype o Singleton) es la mejor opción:

1. Una clase de configuración de reportes con nueve parámetros opcionales, donde casi nunca se usan todos a la vez.
2. Un objeto de configuración ya armado, con datos costosos de calcular, del cual se necesitan variantes menores.
3. Un registro central de auditoría del que debe existir una sola instancia en toda la aplicación.
4. Una función que decide, según el tipo de documento recibido, qué clase concreta de validador instanciar.

5 minutos · respondan en el chat

SÍNTESIS

Los cinco patrones creacionales GoF, completos

Patrón	Idea central
Factory Method	Delegar en una subclase qué clase concreta instanciar
Abstract Factory	Crear familias completas de objetos relacionados sin acoplarse a sus clases concretas
Builder	Separar la construcción paso a paso de un objeto complejo de su representación final
Prototype	Crear objetos nuevos clonando una instancia existente en vez de instanciar desde cero
Singleton	Garantizar una única instancia de una clase, con acceso global controlado (con las reservas vistas hoy)

Con esto se completa el catálogo de creacionales GoF, iniciado el martes con Factory Method y Abstract Factory.



TRANSICIÓN

Ahora, a implementarlos sobre el proyecto real

Los cinco patrones se vieron con ejemplos de clase. El resto de la sesión es el **laboratorio de implementación de patrones creacionales**: cada equipo busca en su propio proyecto de Fábrica Escuela un punto donde la creación de objetos podría beneficiarse de alguno de los cinco patrones vistos (martes + hoy), y lo implementa con código real.



INSTRUCCIONES

Laboratorio: implementación de patrones creacionales

Trabajo en equipo — el mismo equipo del proyecto de Fábrica Escuela conformado desde la semana 2.

- Revisen el código actual de su proyecto y ubiquen **al menos un punto** donde la creación de un objeto sea compleja, repetitiva o poco flexible: constructores con muchos parámetros, instanciación directa de clases concretas, objetos costosos de recrear, un recurso que debería ser único, etc.
- Elijan cuál de los **cinco patrones creacionales** (Factory Method, Abstract Factory, Builder, Prototype o Singleton) resuelve mejor ese punto específico — no hace falta usar más de uno.



INSTRUCCIONES

Implementación y justificación

- Impleméntenlo con **código real** sobre su proyecto — o **pseudocódigo suficientemente detallado** si esa parte del sistema aún no está construida.
- Además del código, escriban una **nota corta** que justifique la elección: por qué ese patrón y no otro de los cinco, y qué problema concreto de creación resolvía en su proyecto.
- Si consideraron un patrón y terminaron usando otro (ej. "pensamos en Factory Method pero el problema real era la cantidad de parámetros opcionales, así que usamos Builder"), vale la pena dejarlo escrito — es evidencia de que la elección fue consciente.

Durante la sesión · trabajo en equipo sobre el proyecto propio

INSTRUCCIONES

Entregable y cierre del laboratorio

- **Formato exacto:** código funcional (o pseudocódigo detallado) que aplique al menos un patrón creacional al proyecto propio, más la nota corta de justificación, en el repositorio o herramienta de gestión del equipo.
- Debe quedar identificable qué **patrón** se aplicó, en qué clase u objeto, y **por qué** se eligió ese y no otro.
- Quien no alcance a terminar durante la sesión puede completarlo **antes de la próxima clase**.

Esta implementación es parte del código que los equipos estarán mostrando en la **Entrega 2 de Fábrica Escuela (15%)**, que se presenta el miércoles de esta semana en el Review Sprint 2 de Coworking — esa sesión no es hoy, pero el trabajo de hoy alimenta directamente esa entrega.

SÍNTESIS

Qué debieron lograr en este laboratorio

1. Identificar un punto real de su proyecto donde la creación de objetos podría mejorar.
2. Elegir, entre los cinco patrones creacionales vistos (martes + hoy), el más adecuado para ese punto.
3. Implementarlo con código real o pseudocódigo suficientemente detallado.
4. Dejar por escrito por qué se eligió ese patrón y no otro de los cinco.



CIERRE

Sigue la segunda familia GoF: los estructurales

Con esto se completa el catálogo de **patrones creacionales GoF** (Factory Method, Abstract Factory, Builder, Prototype, Singleton) aplicado al proyecto propio.

La **semana 13** abre la siguiente familia de la Unidad 5: los **patrones GoF Estructurales** — el martes se ven los primeros, y el viernes los restantes junto con su propio laboratorio de implementación.

